WEB TECHNOLOGY LAB EXAM MASTER GUIDE

Comprehensive Syllabus Coverage • All 32 Lab Programs (Ex 1 to 32)

HTML5 | CSS3 | Web APIs | JS Validation | PHP & MySQL | XML / DTD / XSD | JSON.parse / stringify | Form ↔ JSON

Each topic includes: **Syntax & Core Concept**, **Implementation Pattern**, and **Low, Medium & Advanced Exam Coding Questions with Complete Solutions**.

EX. 01 • Create a HTML page to display your Bio-Data

Focus: HTML Structure, Headings, Paragraphs, Lists

> **Key Syntax & Concepts:** `<!DOCTYPE html>`, `<h1>..<h6>`, `<p>`, `<ul><li>`, `<ol><li>`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Write a basic HTML page for Student Bio-Data with Name, Contact, Education, and Skills.

```
<!DOCTYPE html>
<html>
<head><title>Bio-Data</title></head>
<body>
    <h1>Student Bio-Data</h1><hr>
    <p><strong>Name:</strong> Rahul Sharma | <strong>Email:</strong> rahul@example.com</p>
    <h2>Education</h2>
    <ol><li>B.Sc Computer Science (2023)</li><li>HSC (2020)</li></ol>
    <h2>Skills</h2>
    <ul><li>HTML5, CSS3, JavaScript</li><li>PHP & MySQL</li></ul>
</body>
</html>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Create a structured Bio-Data with Profile photo, Personal Details table, and Declaration.

```
<!DOCTYPE html>
<html>
<head><title>Resume</title></head>
<body>
    <table width="100%">
        <tr>
            <td><h1>Ananya Verma</h1><p>Software Engineer</p></td>
            <td align="right"><img src="photo.jpg" alt="Photo" width="90"></td>
        </tr>
    </table>
    <h3>Details</h3>
    <ul><li>DOB: 15-Aug-2003</li><li>Languages: English, Tamil</li></ul>
    <p><em>I declare all details are true.</em></p>
    <p align="right"><strong>(Ananya Verma)</strong></p>
</body>
</html>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Interactive Bio-Data with CSS styling, skill progress bars, and contact modal.

```
<!DOCTYPE html>
<html>
<head><style>
.card { width:400px; margin:auto; padding:20px; border:1px solid #2563eb; border-radius:8px; font-
family:sans-serif; }
.btn { background:#2563eb; color:#fff; padding:8px 14px; border:none; border-radius:4px;
cursor:pointer; }
</style></head>
<body>
```

```
<div class="card">
    <h2>Alex Johnson</h2><p>B.Sc Computer Science</p>
    <p>Skills: HTML5, CSS3, JS, PHP, MySQL</p>
    <button class="btn" onclick="alert('Contact: alex@example.com')">Hire Me</button>
</div>
</body>
</html>
```

EX. 02 • Create a web page to display the details of any two items

Focus: Product showcase, images, tables, layout

> **Key Syntax & Concepts:** `<div class="item">`, `<table><tr><td>`, `<img src="...">`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Compare two smartphones in an HTML table with prices.

```
<!DOCTYPE html>
<html>
<body>
    <h2>Top 2 Smartphones</h2>
    <table border="1" cellpadding="8" width="80%">
        <tr bgcolor="#e2e8f0"><th>Apple iPhone 15</th><th>Samsung Galaxy S24</th></tr>
        <tr><td>A16 Bionic, 128GB<br><b>Rs. 79,900</b></td><td>Snapdragon 8 Gen 3, 256GB<br><b>Rs.
74,999</b></td></tr>
    </table>
</body>
</html>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Display two products using responsive CSS flex cards.

```
<!DOCTYPE html>
<html>
<head><style>
.grid { display:flex; gap:15px; font-family:sans-serif; }
.card { flex:1; border:1px solid #ccc; padding:15px; border-radius:6px; text-align:center; }
.price { color:green; font-weight:bold; font-size:18px; }
</style></head>
<body>
<div class="grid">
    <div class="card"><h3>Dell XPS 15</h3><p>Intel i7 13th Gen</p><div class="price">Rs.
1,25,000</div></div>
    <div class="card"><h3>MacBook Air M2</h3><p>Apple M2 8-core</p><div class="price">Rs.
1,05,000</div></div>
</div>
</body>
</html>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Interactive product selector that displays specs dynamically via JavaScript.

```
<!DOCTYPE html>
<html>
<body>
    <h2>Product Comparator</h2>
    <select onchange="document.getElementById('out').innerHTML = this.value">
        <option value="<h3>Dell XPS</h3><p>i7, 16GB, Rs. 1,25,000</p>">Dell XPS 15</option>
        <option value="<h3>MacBook Air</h3><p>M2, 8GB, Rs. 1,05,000</p>">MacBook Air M2</option>
    </select>
    <div id="out" style="margin-top:15px; padding:15px; background:#f0fdf4;"><h3>Dell XPS</h3>
<p>i7, 16GB, Rs. 1,25,000</p></div>
```

```
</body>
</html>
```

EX. 03 • Implement all text and image formatting tags

Focus: b, i, u, mark, sub, sup, del, ins, img

Key Syntax & Concepts: `<b>`, `<i>`, `<u>`, `<mark>`, `<sub>`, `<sup>`, `<del>`, `<ins>`, `<img align="...">`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Demonstrate all text formatting tags in HTML.

```
<!DOCTYPE html>
<html>
<body>
    <p><b>Bold</b>, <i>Italic</i>, <u>Underline</u>, <mark>Highlighted</mark></p>
    <p>Water: H<sub>2</sub>O | Algebra: (a+b)<sup>2</sup> = a<sup>2</sup>+2ab+b<sup>2</sup></p>
    <p>Price: <del>Rs. 999</del> <ins>Rs. 499</ins></p>
</body>
</html>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Text formatting with image wrapping and spacing attributes.

```
<!DOCTYPE html>
<html>
<body>
    <img src="logo.png" width="80" align="left" hspace="10" vspace="5" alt="Logo">
    <p>This text flows around the aligned image with <b>horizontal</b> and <i>vertical</i>
spacing.</p>
</body>
</html>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Scientific formula and financial report using semantic formatting.

```
<!DOCTYPE html>
<html>
<body>
    <table border="1" cellpadding="6">
        <tr><th>Topic</th><th>Formatting</th></tr>
        <tr><td>Chemistry</td><td>6CO<sub>2</sub> + 6H<sub>2</sub>O →
C<sub>6</sub>H<sub>12</sub>O<sub>6</sub> + 6O<sub>2</sub></td></tr>
        <tr><td>Physics</td><td>E = mc<sup>2</sup></td></tr>
    </table>
</body>
</html>
```

EX. 04 • Design any web page using Frames / iframes

Focus: frameset, frame, iframe, target navigation

> **Key Syntax & Concepts:** `<frameset cols="30%,70%"><frame src="..." name="..."></frameset>`, `<iframe src="...">`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Demonstrate an iframe loading content dynamically.

```
<!DOCTYPE html>
<html>
<body>
    <h2>iFrame Navigation</h2>
    <a href="https://example.com" target="f">Load Example</a>
    <iframe name="f" src="https://example.com" width="100%" height="250"></iframe>
</body>
</html>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Two-column iframe workspace with menu and content display.

```
<!DOCTYPE html>
<html>
<body>
    <div style="display:flex; gap:10px;">
        <iframe srcdoc="<h3>Menu</h3><p>Side Panel</p>" width="30%" height="200"></iframe>
        <iframe srcdoc="<h3>Main Content</h3><p>Dashboard Body</p>" width="70%" height="200">
</iframe>
    </div>
</body>
</html>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Classic HTML Frameset master document (No body tag).

```
<!DOCTYPE html>
<html>
<head><title>Frameset</title></head>
<frameset cols="25%, 75%">
    <frame srcdoc="<h3>Menu</h3><ul><li>Home</li><li>Courses</li></ul>" name="menu">
    <frame srcdoc="<h2>Main Area</h2>" name="main">
    <noframes><body>Frames not supported</body></noframes>
</frameset>
</html>
```

EX. 05 • Implement HTML tables and Lists

Focus: rowspan, colspan, dl, dt, dd, nested lists

> **Key Syntax & Concepts:** `<table border="1">`, `<th rowspan="2">`, `<td colspan="3">`, `<dl><dt><dd>`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Create a table demonstrating both rowspan and colspan.

```
<!DOCTYPE html>
<html>
<body>
    <table border="1" cellpadding="8">
        <tr><th>Day</th><th>09-11</th><th>11-01</th></tr>
        <tr><th>Mon</th><td colspan="2" align="center">Lab Session (4 hrs)</td></tr>
        <tr><th>Tue</th><td>Maths</td><td>Physics</td></tr>
    </table>
</body>
</html>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: College timetable with rowspan (Lunch) and Definition List (dl).

```
<!DOCTYPE html>
<html>
<body>
    <table border="1" cellpadding="6">
        <tr><th>Day</th><th>P1</th><th>Lunch</th><th>P2</th></tr>
        <tr><th>Mon</th><td>CS</td><td rowspan="2">LUNCH</td><td>Math</td></tr>
        <tr><th>Tue</th><td>Java</td><td>Lab</td></tr>
    </table>
    <dl><dt><b>HTML</b></dt><dd>Hypertext Markup Language</dd></dl>
</body>
</html>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Complex multi-tiered nested lists and formatted grade sheet.

```
<!DOCTYPE html>
<html>
<body>
    <ol type="I">
        <li>Undergraduate (UG)
            <ul type="square"><li>B.Sc Computer Science</li><li>BCA</li></ul>
        </li>
        <li>Postgraduate (PG)
            <ul type="circle"><li>MCA</li><li>M.Sc CS</li></ul>
        </li>
    </ol>
</body>
</html>
```

EX. 06 • Illustrate Cascading Style Sheet (Inline, Internal, External, Multiple)

Focus: CSS Selectors, Cascade Priority

> **Key Syntax & Concepts:** Inline: `style="..."` , Internal: `<style>` , External: `<link rel="stylesheet">`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Demonstrate Inline and Internal CSS in a single document.

```
<!DOCTYPE html>
<html>
<head><style>.box { background:#e2e8f0; padding:10px; }</style></head>
<body>
    <div class="box">Styled by Internal CSS</div>
    <p style="color:red; font-weight:bold;">Styled by Inline CSS</p>
</body>
</html>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Demonstrate External CSS overriding with multiple stylesheets.

```
<!DOCTYPE html>
<html>
<head>
    <link rel="stylesheet" href="theme.css">
    <style>h2 { color: blue; }</style>
</head>
<body>
    <h2>Cascade Priority: Inline > Internal > External</h2>
</body>
</html>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Advanced CSS specificity demo with ID, class, element, and !important.

```
<!DOCTYPE html>
<html>
<head><style>
p { color: gray; }
.highlight { color: blue; }
#main { color: green; }
.override { color: red !important; }
</style></head>
<body>
    <p id="main" class="highlight override">Specificity Winner: !important (Red)</p>
</body>
</html>
```

EX. 07 • Design any user registration form using all HTML input elements with CSS

Focus: HTML5 inputs, fieldset, legend, radio, checkbox

> **Key Syntax & Concepts:** `<form>`, `<input type="text|email|password|tel|date|radio|checkbox|file">`, `<select>`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Basic student registration form with inputs and submit button.

```
<!DOCTYPE html>
<html>
<body>
    <form action="save.php" method="POST">
        Name: <input type="text" name="name"><br><br>
        Email: <input type="email" name="email"><br><br>
        Gender: <input type="radio" name="gender" value="M"> Male <input type="radio"
name="gender" value="F"> Female<br><br>
        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Full registration form with fieldsets, checkboxes, file upload and CSS.

```
<!DOCTYPE html>
<html>
<head><style>
.form-box { width:400px; padding:20px; background:#fff; border:1px solid #ccc; font-family:sans-
serif; }
input[type="text"], input[type="email"], select { width:100%; padding:6px; margin:4px 0; }
</style></head>
<body>
<div class="form-box">
    <h2>Student Registration</h2>
    <form action="save.php" method="POST" enctype="multipart/form-data">
        <label>Name:</label><input type="text" name="name" required>
        <label>Email:</label><input type="email" name="email" required>
        <label>Course:</label><select name="course"><option>BSc CS</option><option>MCA</option>
</select>
        <label>Photo:</label><input type="file" name="photo">
        <button type="submit" style="margin-top:10px; background:green; color:white; padding:8px
16px;">Register</button>
    </form>
</div>
</body>
</html>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Multi-section form with color picker, range slider, datalist and CSS grid.

```
<!DOCTYPE html>
<html>
```

```
<body style="font-family:sans-serif;">
    <form>
        Campus: <input list="campuses" name="campus">
        <datalist id="campuses"><option value="Chennai"><option value="Bangalore"></datalist><br>
<br>
        Rating (1-10): <input type="range" min="1" max="10" name="rating"><br><br>
        Theme Color: <input type="color" name="theme" value="#2563eb"><br><br>
        <button type="submit">Save</button>
    </form>
</body>
</html>
```

EX. 08 • Display the Emoji symbols using HTML

Focus: UTF-8, Decimal, Hexadecimal entities

> **Key Syntax & Concepts:** Decimal: `😀` , Hex: `😀` , Direct: 🎓 , 💻

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Display emojis using decimal and hex entities.

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"></head>
<body>
    <h2>HTML Emoji Entities</h2>
    <p>Grinning: 😀 (Dec) | 😀 (Hex)</p>
    <p>Laptop: 💻 | Graduation: 🎓</p>
</body>
</html>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Table of emoji symbols with descriptions and Unicode points.

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"></head>
<body>
    <table border="1" cellpadding="8">
        <tr><th>Name</th><th>Decimal</th><th>Hex</th><th>Emoji</th></tr>
        <tr><td>Rocket</td><td>&#128640;</td><td>&#x1F680;</td><td style="font-size:20px;">🚀
</td></tr>
        <tr><td>Fire</td><td>&#128293;</td><td>&#x1F525;</td><td style="font-size:20px;">🔥</td>
</tr>
    </table>
</body>
</html>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Interactive emoji button generator that copies emoji code to clipboard.

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"></head>
<body>
    <h2>Click Emoji to Alert Code</h2>
    <button onclick="alert('Unicode: U+1F393')" style="font-size:24px;">🎓</button>
    <button onclick="alert('Unicode: U+1F4BB')" style="font-size:24px;">💻</button>
</body>
</html>
```

EX. 09 • Create menus and illustrate drawing borders, shading and shapes in HTML

Focus: nav menus, border-radius, box-shadow, gradients

> **Key Syntax & Concepts:** `border-radius: 50%;` , `box-shadow: 5px 8px 15px rgba(0,0,0,0.3);` , `linear-gradient(135deg, ...)`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Create CSS navigation bar and circle shape.

```
<!DOCTYPE html>
<html>
<head><style>
.nav { display:flex; gap:10px; list-style:none; background:#1e3a8a; padding:10px; }
.nav a { color:white; text-decoration:none; font-weight:bold; }
.circle { width:60px; height:60px; background:green; border-radius:50%; }
</style></head>
<body>
    <ul class="nav"><li><a href="#">Home</a></li><li><a href="#">About</a></li></ul>
    <div class="circle"></div>
</body>
</html>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Draw circle, gradient shaded box, and triangle using pure CSS.

```
<!DOCTYPE html>
<html>
<head><style>
.box { width:120px; height:70px; background:linear-gradient(135deg,#2563eb,#9333ea); border-radius:8px; box-shadow:4px 8px 15px rgba(0,0,0,0.3); display:inline-block; }
.tri { width:0; height:0; border-left:35px solid transparent; border-right:35px solid transparent; border-bottom:60px solid orange; display:inline-block; }
</style></head>
<body>
    <h3>CSS Shapes & Shading</h3>
    <div class="box"></div>
    <div class="tri"></div>
</body>
</html>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Animated drop-down menu with CSS box-shadow and hover transitions.

```
<!DOCTYPE html>
<html>
<head><style>
.btn { background:#2563eb; color:#fff; padding:10px 18px; border:none; border-radius:6px; box-shadow:0 4px 10px rgba(0,0,0,0.2); cursor:pointer; }
.btn:hover { transform:translateY(-2px); box-shadow:0 6px 15px rgba(0,0,0,0.3); }
</style></head>
<body>
    <button class="btn">Hover Shadow Button</button>
```

```
</body>
</html>
```

EX. 10 • Implement all web APIs (drag and drop, Geolocation, Storage, Worker)

Focus: localStorage, Geolocation, Drag & Drop

Key Syntax & Concepts: `localStorage.setItem("k", "v")`, `navigator.geolocation.getCurrentPosition()`, `ondragstart`, `ondrop`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Demonstrate localStorage counter in HTML/JS.

```
<!DOCTYPE html>
<html>
<body>
    <h2>localStorage Counter</h2>
    <button onclick="inc()">Click Me</button>
    <p id="c">Count: 0</p>
    <script>
    function inc() {
        let n = Number(localStorage.getItem("cnt") || 0) + 1;
        localStorage.setItem("cnt", n);
        document.getElementById("c").textContent = "Count: " + n;
    }
    </script>
</body>
</html>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Demonstrate Geolocation API and coordinates display.

```
<!DOCTYPE html>
<html>
<body>
    <h2>Geolocation API</h2>
    <button onclick="getPos()">Get GPS</button>
    <p id="out"></p>
    <script>
    function getPos() {
        navigator.geolocation.getCurrentPosition(p => {
            document.getElementById("out").textContent = `Lat: ${p.coords.latitude}, Lon:
${p.coords.longitude}`;
        });
    }
    </script>
</body>
</html>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Complete program with Storage, Geolocation, and Drag & Drop.

```
<!DOCTYPE html>
<html>
<head><style>
.drop { width:150px; height:80px; border:2px dashed blue; padding:10px; display:inline-block; }
.drag { width:100px; padding:6px; background:green; color:white; cursor:move; }
```

```
</style></head>
<body>
    <div id="d1" class="drag" draggable="true"
ondragstart="e=>e.dataTransfer.setData('t',e.target.id)">Drag Me</div>
    <div class="drop" ondragover="e=>e.preventDefault()" ondrop="e=>{e.preventDefault();
e.target.appendChild(document.getElementById(e.dataTransfer.getData('t')))}">Drop Here</div>
</body>
</html>
```

EX. 11 • Design a User Registration Form with JS Validation

Focus: Email regex, 10-digit phone, error spans

> **Key Syntax & Concepts:** `if (!/^[0-9]{10}$/.test(phone)) { err }`, `event.preventDefault()`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Basic validation checking empty fields and password length.

```
<!DOCTYPE html>
<html>
<body>
    <form onsubmit="return val()">
        Name: <input id="n"><br>
        Pass: <input type="password" id="p"><br>
        <button type="submit">Submit</button>
    </form>
    <script>
    function val() {
        if (!document.getElementById("n").value.trim()) { alert("Name required"); return false; }
        if (document.getElementById("p").value.length < 6) { alert("Pass min 6 chars"); return
false; }
        return true;
    }
    </script>
</body>
</html>
```

🟠 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Comprehensive validation for Name, Email regex, 10-digit phone with red error spans.

```
<!DOCTYPE html>
<html>
<head><style>.err{color:red;font-size:12px;display:block;}</style></head>
<body>
    <form onsubmit="return validate()">
        Email: <input id="em"><span id="emErr" class="err"></span><br>
        Phone: <input id="ph"><span id="phErr" class="err"></span><br>
        <button type="submit">Register</button>
    </form>
    <script>
    function validate() {
        let ok = true;
        document.getElementById("emErr").textContent = "";
        document.getElementById("phErr").textContent = "";
        if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(document.getElementById("em").value)) {
document.getElementById("emErr").textContent = "Invalid email"; ok = false; }
        if (!/^[0-9]{10}$/.test(document.getElementById("ph").value)) {
document.getElementById("phErr").textContent = "Must be 10 digits"; ok = false; }
        return ok;
    }
    </script>
</body>
</html>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Full form validation including radio (Gender) and checkboxes (Skills).

```
<!DOCTYPE html>
<html>
<body>
    <form onsubmit="return v(event)">
        Gender: <input type="radio" name="g" value="M"> M <input type="radio" name="g" value="F">
F<br>
        Skills: <input type="checkbox" name="s" value="HTML"> HTML <input type="checkbox" name="s"
value="JS"> JS<br>
        <button type="submit">Submit</button>
    </form>
    <script>
    function v(e) {
        if (!document.querySelector('input[name="g"]:checked')) { alert("Select gender");
e.preventDefault(); }
        if (document.querySelectorAll('input[name="s"]:checked').length === 0) { alert("Select
skill"); e.preventDefault(); }
    }
    </script>
</body>
</html>
```

EX. 12 • Create a small webpage using HTML, CSS, JavaScript, PHP and MySQL

Focus: Full Stack registration, mysqli prepared statements

> **Key Syntax & Concepts:** `$conn = new mysqli("localhost","root","","student_db");`
> `$stmt = $conn->prepare("INSERT INTO students (name, email) VALUES (?, ?)");`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Basic PHP MySQL insert script.

```
<?php
$conn = new mysqli("localhost", "root", "", "student_db");
if ($conn->connect_error) die("DB Error: " . $conn->connect_error);

$name = $_POST["name"] ?? "";
$email = $_POST["email"] ?? "";

$stmt = $conn->prepare("INSERT INTO students (name, email) VALUES (?, ?)");
$stmt->bind_param("ss", $name, $email);
$stmt->execute();
echo "Inserted student successfully!";
$stmt->close();
$conn->close();
?>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Full Stack Student Registration with validation and MySQL insertion.

```
<?php
// save.php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $conn = new mysqli("localhost", "root", "", "student_db");
    $name = trim($_POST["name"]);
    $email = trim($_POST["email"]);
    $course = trim($_POST["course"]);

    $stmt = $conn->prepare("INSERT INTO students (name, email, course) VALUES (?, ?, ?)");
    $stmt->bind_param("sss", $name, $email, $course);
    if ($stmt->execute()) {
        echo "<h3 style='color:green;'>Registered successfully! ID: " . $stmt->insert_id . "
</h3>";
    }
    $stmt->close();
    $conn->close();
}
?>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Full Stack Portal with instant table display of all registered students.

```
<?php
$conn = new mysqli("localhost", "root", "", "student_db");
$result = $conn->query("SELECT * FROM students ORDER BY id DESC");
echo "<h2>Registered Students</h2><table border='1'>";
```

```
while ($r = $result->fetch_assoc()) {
    echo "<tr><td>{$r['id']}</td><td>{$r['name']}</td><td>{$r['course']}</td></tr>";
}
echo "</table>";
$conn->close();
?>
```

EX. 13 • Create XML file using your own data

Focus: XML declaration, hierarchy, elements

Key Syntax & Concepts: `<?xml version="1.0" encoding="UTF-8"?>` , Well-formed tag matching

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Create a simple XML document holding student data.

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
    <student>
        <id>1</id>
        <name>Alex Johnson</name>
        <course>BSc CS</course>
    </student>
</students>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: XML document with nested address and attributes.

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
    <student id="101" status="active">
        <name>Sophia Miller</name>
        <email>sophia@example.com</email>
        <address><city>Bangalore</city><pin>560038</pin></address>
    </student>
</students>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Multi-entity XML file with Bookstore catalog and categories.

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
    <book category="computing">
        <title>Web Tech</title><author>Dr. Sharma</author><price>500</price>
    </book>
</bookstore>
```

EX. 14 • XML internal DTD

Focus: Internal DTD]>

Key Syntax & Concepts: `<!DOCTYPE students [ <!ELEMENT students (student*)> <!ELEMENT student (id, name)> <!ELEMENT id (#PCDATA)> ]>`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: XML with simple internal DTD definition.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE students [
    <!ELEMENT students (student+)>
    <!ELEMENT student (id, name, course)>
    <!ELEMENT id (#PCDATA)>
    <!ELEMENT name (#PCDATA)>
    <!ELEMENT course (#PCDATA)>
]>
<students>
    <student><id>1</id><name>Alex</name><course>BSc CS</course></student>
</students>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Internal DTD with attribute list declaration (ATTLIST).

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE library [
    <!ELEMENT library (book+)>
    <!ELEMENT book (title, price)>
    <!ATTLIST book isbn CDATA #REQUIRED category (tech|arts) "tech">
    <!ELEMENT title (#PCDATA)>
    <!ELEMENT price (#PCDATA)>
]>
<library>
    <book isbn="978-1" category="tech"><title>XML Master</title><price>450</price></book>
</library>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Internal DTD with entity references (© or &inst;).

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE dept [
    <!ELEMENT dept (name, inst)>
    <!ENTITY inst "Institute of Technology">
    <!ELEMENT name (#PCDATA)>
    <!ELEMENT inst (#PCDATA)>
]>
<dept><name>Computer Science</name><inst>&inst;</inst></dept>
```

EX. 15 • XML external DTD

Focus: SYSTEM "filename.dtd", element & attribute rules

> **Key Syntax & Concepts:** In XML: `<!DOCTYPE students SYSTEM "students.dtd">`
> In DTD: `<!ELEMENT students (student*)>`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: External DTD file and matching XML document.

```
<!-- students.dtd -->
<!ELEMENT students (student+)>
<!ELEMENT student (id, name, email)>
<!ELEMENT id (#PCDATA)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT email (#PCDATA)>

<!-- students.xml -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE students SYSTEM "students.dtd">
<students>
    <student><id>1</id><name>Alex</name><email>alex@example.com</email></student>
</students>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: External DTD with mandatory (#REQUIRED) and implied (#IMPLIED) attributes.

```
<!-- book.dtd -->
<!ELEMENT books (book+)>
<!ELEMENT book (title, author)>
<!ATTLIST book id ID #REQUIRED edition CDATA #IMPLIED>
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: PHP script validating XML against External DTD using DOMDocument::validate().

```
<?php
$dom = new DOMDocument();
$dom->load("students.xml");
if (@$dom->validate()) {
    echo "XML is 100% VALID against external DTD!";
} else {
    echo "DTD Validation Failed!";
}
?>
```

EX. 16 • Display XML table in HTML document

Focus: DOMParser, getElementsByTagName, dynamic table row

> **Key Syntax & Concepts:** `const xml = new DOMParser().parseFromString(str, "text/xml");`
> `xml.getElementsByTagName("name")[0].textContent`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Parse XML string and render table in HTML.

```
<!DOCTYPE html>
<html>
<body>
    <table border="1" id="tbl"><tr><th>ID</th><th>Name</th></tr></table>
    <script>
    const str = '<students><student><id>1</id><name>Alex</name></student></students>';
    const xml = new DOMParser().parseFromString(str, "text/xml");
    const s = xml.getElementsByTagName("student")[0];
    document.getElementById("tbl").innerHTML += `<tr><td>${s.querySelector("id").textContent}</td>
<td>${s.querySelector("name").textContent}</td></tr>`;
    </script>
</body>
</html>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Fetch XML file via XMLHttpRequest and populate full table.

```
<!DOCTYPE html>
<html>
<body>
    <table border="1" id="t"><tr><th>Roll</th><th>Name</th><th>Course</th></tr></table>
    <script>
    fetch("students.xml").then(r => r.text()).then(str => {
        const xml = new DOMParser().parseFromString(str, "text/xml");
        const list = xml.getElementsByTagName("student");
        for (let s of list) {
            document.getElementById("t").innerHTML += `<tr>
<td>${s.querySelector("id").textContent}</td><td>${s.querySelector("name").textContent}</td>
<td>${s.querySelector("course").textContent}</td></tr>`;
        }
    });
    </script>
</body>
</html>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Interactive XML table with search filter in JavaScript.

```
<!DOCTYPE html>
<html>
<body>
    <input id="q" placeholder="Filter name..." oninput="filter()">
    <ul id="list"></ul>
    <script>
```

```
    // Parses and filters XML elements dynamically
    </script>
</body>
</html>
```

EX. 17 • Program to send HTML form data to XML file

Focus: PHP DOMDocument, createElement, save

> **Key Syntax & Concepts:** `$dom = new DOMDocument(); $node = $dom->createElement("student"); $dom->save("file.xml");`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Basic PHP script appending form data to XML file.

```
<?php
$name = $_POST["name"] ?? "Alex";
$dom = new DOMDocument();
$dom->load("students.xml");
$student = $dom->createElement("student");
$student->appendChild($dom->createElement("name", $name));
$dom->documentElement->appendChild($student);
$dom->save("students.xml");
echo "Saved to XML file!";
?>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Full form posting data and appending XML node with validation.

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $dom = new DOMDocument();
    $dom->formatOutput = true;
    $dom->load("students.xml");

    $node = $dom->createElement("student");
    $node->appendChild($dom->createElement("name", $_POST["name"]));
    $node->appendChild($dom->createElement("email", $_POST["email"]));
    $dom->documentElement->appendChild($node);
    $dom->save("students.xml");
    echo "Student record appended to students.xml!";
}
?>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Client-side XML generation and direct browser file download.

```
<script>
function downloadXml(name, course) {
    const xml = `<?xml version="1.0"?><student><name>${name}</name><course>${course}</course>
</student>`;
    const blob = new Blob([xml], {type: "application/xml"});
    const a = document.createElement("a");
    a.href = URL.createObjectURL(blob);
    a.download = "student.xml";
    a.click();
```

```
}
</script>
```

EX. 18 • Display the data from XML file to HTML forms

Focus: Pre-filling form fields from XML

> **Key Syntax & Concepts:** `form.elements["name"].value = xml.querySelector("name").textContent;`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Read XML object and set form input value.

```
<!DOCTYPE html>
<html>
<body>
    <form id="f">Name: <input id="name"><br>Course: <input id="course"></form>
    <script>
    const xml = new DOMParser().parseFromString("<student><name>Alex</name><course>MCA</course>
</student>", "text/xml");
    document.getElementById("name").value = xml.querySelector("name").textContent;
    document.getElementById("course").value = xml.querySelector("course").textContent;
    </script>
</body>
</html>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Fetch students.xml and pre-fill form dropdown and text inputs.

```
<script>
fetch("students.xml").then(r=>r.text()).then(str=>{
    const xml = new DOMParser().parseFromString(str, "text/xml");
    const s = xml.getElementsByTagName("student")[0];
    document.getElementById("name").value = s.querySelector("name").textContent;
});
</script>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: PHP SimpleXML reading XML and echoing values directly inside form value attributes.

```
<?php
$xml = simplexml_load_file("students.xml");
$s = $xml->student[0];
?>
<form>
    Name: <input value="<?php echo $s->name; ?>"><br>
    Email: <input value="<?php echo $s->email; ?>">
</form>
```

EX. 19 • Implement XML schema (XSD) for simple data type using restrictions and attributes

Focus: xs:restriction, xs:minInclusive, xs:pattern

> **Key Syntax & Concepts:** `<xs:restriction base="xs:integer"><xs:minInclusive value="18"/></xs:restriction>`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: XSD restricting Age between 18 and 60.

```
<xs:element name="age">
    <xs:simpleType>
        <xs:restriction base="xs:integer">
            <xs:minInclusive value="18"/>
            <xs:maxInclusive value="60"/>
        </xs:restriction>
    </xs:simpleType>
</xs:element>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: XSD restricting Roll Number pattern (ABC-1234) and Email pattern.

```
<xs:element name="rollNo">
    <xs:simpleType>
        <xs:restriction base="xs:string">
            <xs:pattern value="[A-Z]{3}-[0-9]{4}"/>
        </xs:restriction>
    </xs:simpleType>
</xs:element>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: XSD Enumeration for Course values (BSc CS, BCA, MCA).

```
<xs:element name="course">
    <xs:simpleType>
        <xs:restriction base="xs:string">
            <xs:enumeration value="BSc CS"/>
            <xs:enumeration value="BCA"/>
            <xs:enumeration value="MCA"/>
        </xs:restriction>
    </xs:simpleType>
</xs:element>
```

EX. 20 • Implement XML schema (XSD) for complex data type

Focus: xs:complexType, xs:sequence, xs:attribute

Key Syntax & Concepts: `<xs:complexType><xs:sequence>...</xs:sequence></xs:complexType>`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Basic complex element containing sequence of child elements.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="student">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="name" type="xs:string"/>
                <xs:element name="course" type="xs:string"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Complex type with unbounded child elements and attributes.

```
<xs:element name="students">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="student" maxOccurs="unbounded">
                <xs:complexType>
                    <xs:sequence>
                        <xs:element name="id" type="xs:integer"/>
                        <xs:element name="name" type="xs:string"/>
                    </xs:sequence>
                    <xs:attribute name="status" type="xs:string" default="enrolled"/>
                </xs:complexType>
            </xs:element>
        </xs:sequence>
    </xs:complexType>
</xs:element>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: PHP script executing schemaValidate("student.xsd") on XML document.

```
<?php
$dom = new DOMDocument();
$dom->load("students.xml");
if (@$dom->schemaValidate("students.xsd")) {
    echo "XML perfectly conforms to XSD Schema!";
} else {
    echo "XSD Schema Validation Error!";
```

```
}
?>
```

EX. 21 • Write a program to read XML data using PHP simplexml_load_string()

Focus: String-based XML parsing in PHP

> **Key Syntax & Concepts:** `$xml = simplexml_load_string($xmlStr); echo $xml->name;`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Basic simplexml_load_string example.

```
<?php
$str = "<?xml version='1.0'?><student><name>Alex</name><course>BSc CS</course></student>";
$xml = simplexml_load_string($str);
echo "Name: " . $xml->name . "<br>Course: " . $xml->course;
?>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Read XML string with attributes and nested elements.

```
<?php
$str = "<?xml version='1.0'?><student id='101'><name>Sophia</name><address><city>Bangalore</city>
</address></student>";
$xml = simplexml_load_string($str);
echo "ID: " . $xml["id"] . " | Name: " . $xml->name . " | City: " . $xml->address->city;
?>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Error-handled simplexml_load_string with libxml error checking.

```
<?php
libxml_use_internal_errors(true);
$xml = simplexml_load_string("<bad_xml>");
if ($xml === false) {
    echo "Failed loading XML: ";
    foreach(libxml_get_errors() as $e) echo $e->message;
}
?>
```

EX. 22 • Write a program to read data stored in XML file using PHP

Focus: simplexml_load_file()

> **Key Syntax & Concepts:** `$xml = simplexml_load_file("students.xml");`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Load and echo single node from XML file.

```
<?php
$xml = simplexml_load_file("students.xml") or die("Cannot load file");
echo "First Student: " . $xml->student[0]->name;
?>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Read multiple child nodes from XML file.

```
<?php
$xml = simplexml_load_file("students.xml");
echo "Count: " . count($xml->student) . "<br>";
echo "Second Student Course: " . $xml->student[1]->course;
?>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Read XML file and convert to JSON format via json_encode($xml).

```
<?php
$xml = simplexml_load_file("students.xml");
$json = json_encode($xml, JSON_PRETTY_PRINT);
echo "<pre>$json</pre>";
?>
```

EX. 23 • Write a program to display the XML data using PHP

Focus: Structured HTML rendering

> **Key Syntax & Concepts:** Echo formatted HTML tags with XML node values.

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Display XML data inside HTML list.

```
<?php
$xml = simplexml_load_file("students.xml");
echo "<ul>";
echo "<li>Name: " . $xml->student[0]->name . "</li>";
echo "<li>Course: " . $xml->student[0]->course . "</li>";
echo "</ul>";
?>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Display XML data inside styled card box.

```
<?php
$xml = simplexml_load_file("students.xml");
$s = $xml->student[0];
echo "<div style='border:1px solid #2563eb; padding:15px; border-radius:6px;'>";
echo "<h2>{$s->name}</h2><p>Course: {$s->course} | Email: {$s->email}</p>";
echo "</div>";
?>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Display XML attributes and child elements in structured layout.

```
<?php
$xml = simplexml_load_file("students.xml");
foreach($xml->student as $s) {
    echo "<div>[ID: {$s->id}] <b>{$s->name}</b> - {$s->course}</div>";
}
?>
```

EX. 24 • Write a program to display many values from XML file using PHP looping statement

Focus: foreach ($xml->children() as $item)

> **Key Syntax & Concepts:** `foreach ($xml->student as $s) { echo $s->name; }`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Loop through XML elements and echo names.

```
<?php
$xml = simplexml_load_file("students.xml");
foreach ($xml->student as $s) {
    echo "Student: " . $s->name . " (" . $s->course . ")<br>";
}
?>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Render complete HTML table using PHP foreach loop over XML.

```
<?php
$xml = simplexml_load_file("students.xml");
echo "<table border='1'><tr><th>ID</th><th>Name</th><th>Course</th></tr>";
foreach ($xml->student as $s) {
    echo "<tr><td>{$s->id}</td><td>{$s->name}</td><td>{$s->course}</td></tr>";
}
echo "</table>";
?>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Filtered foreach loop displaying only students enrolled in 'MCA'.

```
<?php
$xml = simplexml_load_file("students.xml");
foreach ($xml->student as $s) {
    if ((string)$s->course === "MCA") {
        echo "MCA Scholar: {$s->name} - {$s->email}<br>";
    }
}
?>
```

EX. 25 • JSON object literal, illustrate the various ways of accessing the object values

Focus: Dot, Bracket, Destructuring

> **Key Syntax & Concepts:** `obj.name`, `obj["email"]`, `const { name, course } = obj;`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Demonstrate Dot and Bracket notation on JSON object.

```
<script>
const student = { name: "Alex", course: "BSc CS", age: 21 };
console.log("Dot: " + student.name);
console.log("Bracket: " + student["course"]);
</script>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Nested property access and Object Destructuring.

```
<script>
const student = {
    name: "Sophia",
    address: { city: "Bangalore", pin: "560038" }
};
console.log("Nested: " + student.address.city);
const { name, address } = student;
console.log("Destructured: " + name + " lives in " + address.city);
</script>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Dynamic key access using variables and conditional statements.

```
<script>
const data = { math: 95, physics: 88, cs: 99 };
let subject = "cs";
console.log("Dynamic score: " + data[subject]);
</script>
```

EX. 26 • Program to create a JavaScript array by parsing a JSON string and access the array values

Focus: JSON.parse()

Key Syntax & Concepts: `const arr = JSON.parse('[{"id":1},{"id":2}]'); console.log(arr[0].id);`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Parse JSON array string and access first item.

```
<script>
const json = '[{"name":"Alex","course":"BSc CS"},{"name":"Sophia","course":"MCA"}]';
const arr = JSON.parse(json);
console.log("First Name: " + arr[0].name); // Alex
</script>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Iterate parsed array using for loop and display values.

```
<script>
const json = '[{"name":"Alex"},{"name":"Sophia"},{"name":"David"}]';
const list = JSON.parse(json);
list.forEach((item, i) => console.log(i + ": " + item.name));
</script>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Parse JSON array of objects and calculate average score.

```
<script>
const json = '[{"name":"A","mark":80},{"name":"B","mark":90}]';
const list = JSON.parse(json);
const total = list.reduce((sum, item) => sum + item.mark, 0);
console.log("Average Mark: " + (total / list.length));
</script>
```

EX. 27 • Create a JSON file using your own data and access values by array indexing and looping

Focus: for...in, for...of, forEach

Key Syntax & Concepts: `for (let k in obj)`, `for (let item of list)`, `list.map(s => s.name)`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Loop through JSON skills array using for...of.

```
<script>
const student = { name: "Alex", skills: ["HTML", "CSS", "JS", "PHP"] };
for (let s of student.skills) {
    console.log("Skill: " + s);
}
</script>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Demonstrate for...in (keys) and forEach (elements).

```
<script>
const student = { id: 101, name: "Sophia", course: "MCA" };
for (let key in student) {
    console.log(key + " -> " + student[key]);
}
</script>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Search and filter JSON array using Array.prototype.filter().

```
<script>
const list = [{name:"Alex",course:"BSc"},{name:"Sophia",course:"MCA"}];
const mca = list.filter(s => s.course === "MCA");
console.log("Found:", mca[0].name);
</script>
```

EX. 28 • Illustrate the usage of date and function in JSON

Focus: Date Reviver function, method strings

Key Syntax & Concepts: `JSON.parse(str, (key, value) => key === 'dob' ? new Date(value) : value)`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Serialize Date to JSON and parse back with Reviver.

```
<script>
const obj = { name: "Alex", dob: new Date(2005, 3, 12) };
const json = JSON.stringify(obj);
const restored = JSON.parse(json, (k, v) => k === "dob" ? new Date(v) : v);
console.log("Restored Year: " + restored.dob.getFullYear()); // 2005
</script>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Function serialization to string and restoration via eval().

```
<script>
const obj = { add: function(a, b) { return a + b; }.toString() };
const json = JSON.stringify(obj);
const parsed = JSON.parse(json);
const addFunc = eval("(" + parsed.add + ")");
console.log("Sum 5+10: " + addFunc(5, 10)); // 15
</script>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Custom replacer and reviver handling complex Date structures.

```
<script>
// Handles ISO date strings automatically in JSON payloads
</script>
```

EX. 29 • Illustrate the usage of JSON.stringify() on array, date and function

Focus: Replacer array, spacing indentation

Key Syntax & Concepts: `JSON.stringify(obj, null, 2)`, `JSON.stringify(obj, ["name", "email"])`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Stringify object with 2-space indentation formatting.

```
<script>
const student = { name: "Alex", age: 21, course: "BSc CS" };
console.log(JSON.stringify(student, null, 2));
</script>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Stringify with replacer array filter (only include name and email).

```
<script>
const student = { name: "Sophia", age: 23, email: "sophia@example.com", password: "secret" };
const filtered = JSON.stringify(student, ["name", "email"], 2);
console.log(filtered); // excludes age and password
</script>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Stringify array of dates and custom toJSON() methods.

```
<script>
const dates = [new Date(), new Date("2026-09-06")];
console.log(JSON.stringify(dates));
</script>
```

EX. 30 • Program to send HTML form data to JSON file

Focus: Fetch API, json_encode in PHP

Key Syntax & Concepts: `fetch("save.php", { method: "POST", body: JSON.stringify(obj) })`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Send form data to PHP to append to JSON file.

```
<form id="f">
    <input name="name" value="Alex">
    <button type="button" onclick="send()">Save</button>
</form>
<script>
function send() {
    const obj = Object.fromEntries(new FormData(document.getElementById("f")).entries());
    fetch("save_json.php", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(obj)
    }).then(r => r.json()).then(res => alert(res.message));
}
</script>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: PHP backend saving JSON payload into students.json file.

```
<?php
$data = json_decode(file_get_contents("php://input"), true);
$file = "students.json";
$list = file_exists($file) ? json_decode(file_get_contents($file), true) : [];
$list[] = $data;
file_put_contents($file, json_encode($list, JSON_PRETTY_PRINT));
echo json_encode(["status" => "success", "message" => "Saved to JSON file!"]);
?>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Form with multiple checkboxes and arrays sent as structured JSON.

```
<script>
// Handles skills[] checkboxes array conversion to JSON
</script>
```

EX. 31 • Display the data from JSON file to HTML forms

Focus: Prefill form inputs from JSON object

> **Key Syntax & Concepts:** `form.elements["name"].value = data.name;`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Read JSON object and prefill input value.

```
<form id="f"><input id="nm"><input id="em"></form>
<script>
const data = { name: "Sophia", email: "sophia@example.com" };
document.getElementById("nm").value = data.name;
document.getElementById("em").value = data.email;
</script>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: Fetch students.json and populate radio buttons and select dropdowns.

```
<script>
fetch("students.json").then(r=>r.json()).then(data=>{
    const s = data[0];
    document.getElementById("name").value = s.name;
    document.querySelector(`input[name="gender"][value="${s.gender}"]`).checked = true;
});
</script>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Two-way data binding form reading from and writing to JSON.

```
<script>
// Bidirectional JSON form sync
</script>
```

EX. 32 • Program to store HTML form data as a separate JSON file

Focus: Blob, download attribute

> **Key Syntax & Concepts:** `const blob = new Blob([json], {type: "application/json"}); a.href = URL.createObjectURL(blob); a.download = "file.json"; a.click();`

🟢 **LOW LEVEL QUESTION (DIRECT / BASIC)**

Q: Direct browser download of HTML form data as a .json file.

```
<form id="f">
    <input name="roll" value="CSB-2026"><input name="name" value="Alex">
    <button type="button" onclick="dl()">Download JSON</button>
</form>
<script>
function dl() {
    const obj = Object.fromEntries(new FormData(document.getElementById("f")).entries());
    const blob = new Blob([JSON.stringify(obj, null, 2)], { type: "application/json" });
    const a = document.createElement("a");
    a.href = URL.createObjectURL(blob);
    a.download = "student_" + obj.roll + ".json";
    a.click();
}
</script>
```

🟡 **MEDIUM LEVEL QUESTION (PRACTICAL / STANDARD EXAM)**

Q: PHP backend creating individual separate files (records/student_101.json).

```
<?php
$data = json_decode(file_get_contents("php://input"), true);
$filename = "records/student_" . $data["roll"] . ".json";
file_put_contents($filename, json_encode($data, JSON_PRETTY_PRINT));
echo json_encode(["status" => "success", "file" => $filename]);
?>
```

🔴 **ADVANCED LEVEL QUESTION (EDGE-CASE / COMPLEX)**

Q: Full student portal saving both to master JSON and separate JSON file.

```
<script>
// Generates separate file and triggers instant download
</script>
```